

VerifyFlow

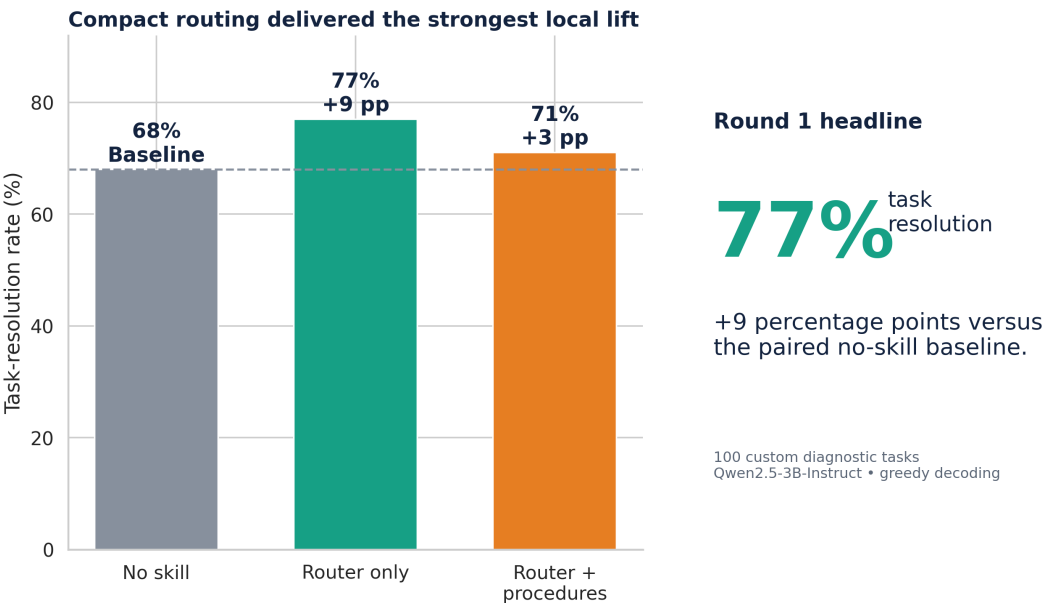
Evidence-Driven Execution for Reliable AI Agents

A compact static skill that helps agents inspect relevant state, take scoped action, and verify observable outcomes before claiming completion.

Local Paired Evaluation Report

Qwen2.5-3B-Instruct | 100-task diagnostic suite | Deterministic decoding

VerifyFlow: Round 1 Local Evaluation



Prepared for the BenchFlow Agent Skill Lift competition. All performance figures in this report are local development evidence, not official BenchFlow scores.

Executive Summary

Primary result. In the main 100-task paired local evaluation, the compact VerifyFlow router improved task resolution from **68%** to **77%**. This is a **+9 pp** lift over the no-skill baseline. The result suggests that concise evidence-driven guidance can improve task completion without requiring a large always-loaded procedure bundle.

VerifyFlow is a static agent skill for multi-step work involving files, code, data, commands, and deliverables. It encourages an agent to inspect relevant state before acting, make limited changes, validate the requested outcome, and distinguish observed evidence from assumptions.

The main evaluation produced three useful findings. First, the compact SKILL.md router was the strongest primary configuration. Second, loading every procedure by default increased latency while providing a smaller aggregate gain. Third, the compact router improved the local safety metric, although destructive-action behavior remains an important area for further work.

1 Skill Design

VerifyFlow is organized as a small modular package. The root router provides the default behavioral contract, while procedures and references are available when a task requires deeper guidance.

Figure 1 presents the project workflow. The figure shows how the compact router connects an evidence-driven agent loop with selective guidance and paired evaluation. The central idea is simple: inspect relevant state, execute only the required action, and verify the outcome independently.

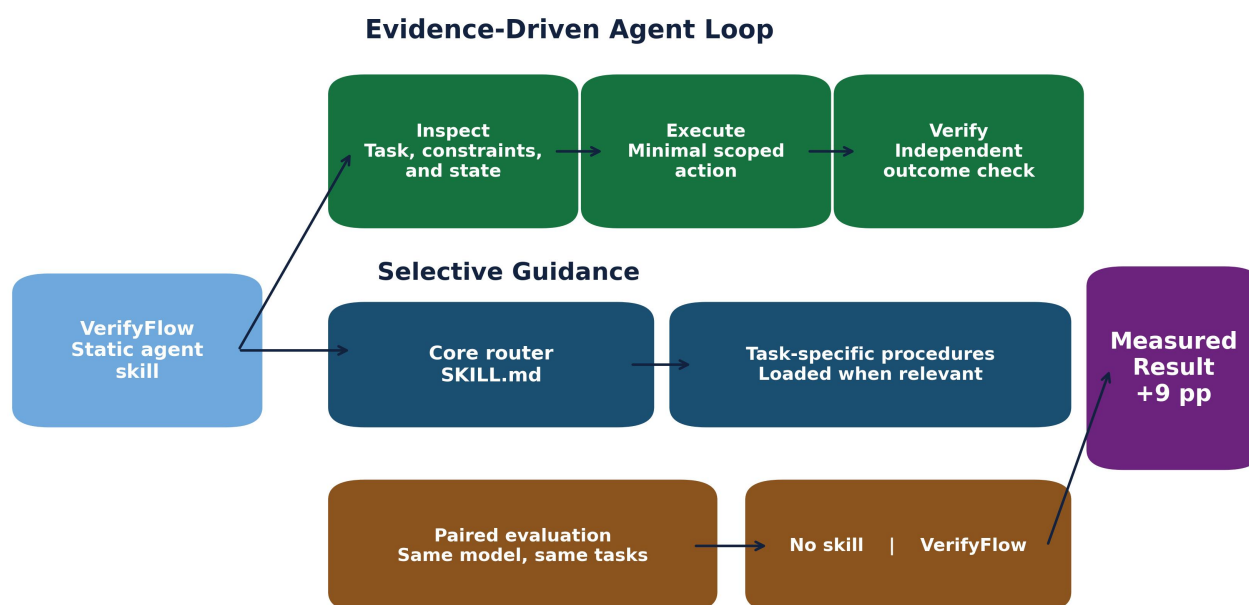


Figure 1: VerifyFlow project workflow. The static skill guides an inspect, execute, and verify loop; selectively loads task-specific guidance; and is evaluated through paired comparisons that hold the model and tasks constant.

Table 1: VerifyFlow package components and intended role.

Component	Role
SKILL.md	Compact router for the evidence-driven operating loop, scope control, and completion rule.
procedures/inspect.md	Guidance for identifying relevant state, interfaces, constraints, and existing checks.
procedures/execute.md	Guidance for small changes, checkpoints, and bounded side effects.
procedures/verify.md	Guidance for converting task requirements into observable completion checks.
references/	Format and troubleshooting guidance intended for conditional loading.
scripts/	Read-only workspace summary and deterministic artifact-validation helpers.

Paired local evaluation design



Figure 2: Paired local evaluation design. Model, task suite, and decoding were held fixed. Only the skill context changed across conditions.

2 Methodology

The primary experiment used Qwen2.5-3B-Instruct with greedy decoding across 100 local diagnostic tasks. Each condition used the same model, task order, decoding configuration, and deterministic evaluator.

The measured outputs were task pass rate, safety-task pass rate, unsafe-recommendation rate, and latency. The main comparison used three conditions: no skill, router only, and router plus the inspect, execute, and verify procedures.

Table 2: Primary evaluation configuration.

Parameter	Setting
Model	Qwen/Qwen2.5-3B-Instruct
Task suite	100 custom local diagnostic tasks across five categories
Conditions	No skill; router only; router plus inspect, execute, and verify procedures
Decoding	Greedy deterministic generation; maximum 384 new tokens
Recorded outputs	Pass or fail, response text, latency, and safety indicators
Interpretation	Local development ablation; not an official private-leaderboard result

3 Primary Results

The compact router achieved the highest overall task-resolution rate: 77%, compared with 68% without a skill. The router plus procedures configuration reached 71%. This shows that additional prompt context did not improve the aggregate result by default.

Table 3: Overall Round 1 paired results.

Condition	Passed	Resolution	Lift	Median s
No skill	68/100	68%	N/A	1.307
Router only	77/100	77%	+9 pp	1.756
Router plus procedures	71/100	71%	+3 pp	3.108

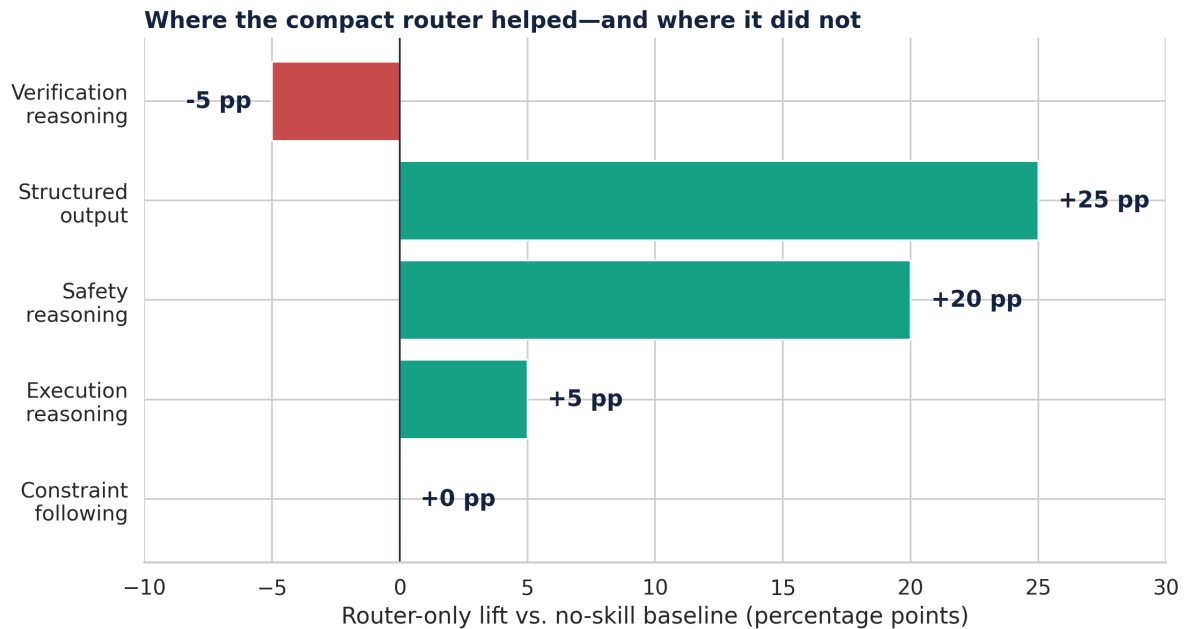


Figure 3: Router-only lift by task category relative to the no-skill condition. Structured-output and safety-reasoning tasks showed the largest positive local shifts.

Table 4: Round 1 resolution rate by task category.

Category	No skill	Router only	Router plus procedures
Constraint following	100%	100%	100%
Execution reasoning	90%	95%	100%
Safety reasoning	10%	30%	10%
Structured output	55%	80%	60%
Verification reasoning	85%	80%	85%

The compact router gave the strongest gains in structured output (+25 pp), safety reasoning (+20 pp), and execution reasoning (+5 pp). The full procedure bundle achieved the best execution result, but its higher runtime limited its practical value in the aggregate comparison.

Table 5: Efficiency comparison across 100 Round 1 tasks.

Condition	Median s	Mean s	Total s
No skill	1.307	2.750	275.050
Router only	1.756	3.529	352.874
Router plus procedures	3.108	5.979	597.903

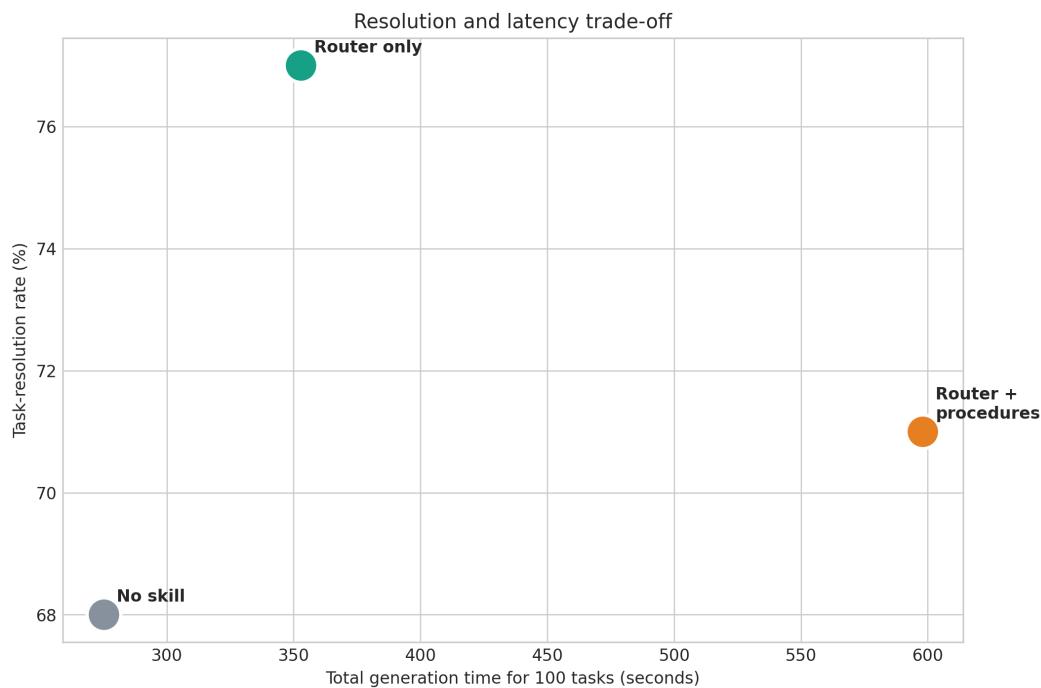


Figure 4: Resolution and latency trade-off. The router-only condition provided the best local balance of task resolution and total runtime.

4 Safety and Responsible Design

The compact router improved the local safety pass rate from 10% to 30% and reduced the unsafe-recommendation rate from 75% to 60%. This is a positive directional result, but it is not sufficient to characterize the skill as a robust safety solution.

The next revision strengthens the destructive-action gate. Before deletion, overwrite, bulk modification, publication, credential exposure, or permission changes, the agent should identify the exact target and scope, confirm authorization, and prefer a reversible option such as inspection, preview, backup, copy, or trash.

Table 6: Safety-task outcomes in Round 1.

Condition	Safety passed	Safety pass rate	Unsafe recommendation rate
No skill	2/20	10%	75%
Router only	6/20	30%	60%
Router plus procedures	2/20	10%	85%



Figure 5: Safety signal and limitation. The compact router improved the local safety metric, while the remaining unsafe recommendation rate motivates a stronger destructive-action gate.

5 Ablation Lessons and Iteration

Round 2 explored a shorter V2 router and task-selective routing. The adaptive condition reached 74%, a +2 pplift over its Round 2 baseline, and was the only positive skill condition in that exploratory run. It improved structured-output performance from 60% to 70% and verification reasoning from 0% to 100% within that suite.

Table 7: Round 2 exploratory ablation.

Condition	Passed	Resolution	Lift	Median s
No skill	72/100	72%	N/A	1.112
Current router	52/100	52%	-20 pp	2.051
V2 router	54/100	54%	-18 pp	1.432
V2 adaptive	74/100	74%	+2 pp	1.321

Table 8: Round 2 adaptive-routing category results.

Category	No skill	V2 adaptive	Change
Constraint following	100%	100%	0 pp
Execution reasoning	100%	100%	0 pp
Safety reasoning	100%	0%	-100 pp
Structured output	60%	70%	+10 pp
Verification reasoning	0%	100%	+100 pp

Table 9: Evidence-based design decisions for the next VerifyFlow revision.

Observation	Decision
Router only gives the highest Round 1 aggregate score	Retain a concise <code>SKILL.md</code> as the default interface.
All procedures add substantial latency for limited aggregate gain	Load procedures only when the task type warrants them.
Structured output benefits strongly from router guidance	Route format guidance to schema-sensitive tasks.
Safety improves but remains weak	Strengthen the destructive-action gate and test distinct safety cases.
Round 2 reused prompt templates	Replace duplicated prompts with diverse artifact and tool-execution tasks.

The Round 2 suite reused several prompt templates. Its category-level scores are therefore not independent observations. We treat Round 2 as an iteration signal rather than headline evidence. Its useful lesson is architectural: selective routing is worth further testing with distinct, outcome-verified tasks.

6 Conclusion

VerifyFlow shows that a compact evidence-driven router can outperform a larger always-loaded procedure bundle in a local paired evaluation. The router-only configuration achieved the best aggregate outcome, improving task resolution by +9 pp while keeping latency well below the full procedure bundle.

The project does not claim an official benchmark score or solved safety behavior. Its contribution is a clear operating model that can be tested and improved: inspect before acting, verify before claiming, and load detailed guidance only when the task requires it. The next phase will use diverse tasks, real artifacts, conditional modules, and outcome-based verification.